

ConvMgr User Manual

ConvMgr Application Notes
November 25, 2020

Advantest Europe GmbH
Branch / Zweigniederlassung Boeblingen
Herrenberger Str. 130
D-71034 Boeblingen

Contents

1	Introduction	1
1.1	Audience	1
1.2	Concept	1
1.3	Methodology	1
2	Installation	2
3	Invocation	3
3.1	Using Command Line	3
3.1.1	Synopsys	3
3.1.2	Command Line Option Description	3
3.1.3	An Example of the Command Line	3
3.1.4	An Example of Template Generation	4
4	Generating Advantest V93000 Output from STIL	5
4.1	Test Program Description Spreadsheet	5
4.1.1	Global Sheet	6
4.1.2	TestList Sheet	7
4.1.3	Conversion Setup sheet	9
4.1.4	TestSuite Flags	10
4.1.5	TestMethod Sheet	11
4.1.6	Pins sheet	11
4.1.7	Ports Sheet and Groups Sheet	11
4.1.8	PortConfig Sheet	12
4.1.9	Bins sheet	12
4.1.10	TestFlow Flags	14
4.1.11	TestFlow Variables	14
4.1.12	Tests	14
4.1.13	Utility Purposes	15
4.2	TestFlow Skeleton	15
4.3	Output Files	17
4.3.1	Test Program Directory	17
4.3.2	Reference	18
5	Generating Advantest V93000SMT8 Output from STIL	19
5.1	Test Program Description Spreadsheet	19
5.1.1	Global Sheet	19
5.1.2	TestList Sheet	21
5.1.3	Conversion Setup sheet	23
5.1.4	TestMethod Sheet	24
5.1.5	Pins sheet	24
5.1.6	Ports Sheet and Groups Sheet	24

5.1.7	PortConfig Sheet	25
5.1.8	Bins sheet	25
5.2	Flow Skeleton	26
5.3	Output Files	27
5.3.1	Test Program Directory	27
5.3.2	Reference	28
6	Additional Capabilities	29
6.1	Import Pins	29
6.1.1	Example of Import Pin Usage	29
6.2	Suggest Pattern Groups	29
6.2.1	Example of Suggest Pattern Groups Usage	29
6.3	Check Project	30
6.3.1	Example of Check Project Usage	30

Chapter 1

Introduction

1.1 Audience

ConvMgr User Manual is intended for test and design engineers of integrated circuits.

1.2 Concept

ConvMgr is designed to manage translation of a number of STIL files into a single test program. Each STIL file is attributed to one or more test suite/instance in the testflow. Basically, ConvMgr can be viewed as a wrapper around STILReader.

1.3 Methodology

ConvMgr provides a complete flow from a list of STIL patterns into a single test program. The conversion of the STIL patterns and test program assembly is managed using an Excel spreadsheet, consisting of several sheet (such as tests list sheet, setup flags sheet, pins sheet etc.), that form the entire test program data.

ConvMgr operational flow is as follows:

1. Prepare setup and auxiliary files (such as external levels files) for every STIL pattern in the spreadsheet.
2. Execute STILReader on every STIL pattern, using setup files from the previous step.
3. Assemble the test program by copying the binary patterns, combining the timing and levels specifications into one or more files (according to the ConvMgr configuration and target platform), creating additional device files based on the spreadsheet (such as pins file), and generating the main flow file.

Chapter 2

Installation

ConvMgr is a part of STDL Toolset and is installed together with other TDL Toolset applications. For more information, see STDL Toolset Installation Guide.

Chapter 3

Invocation

3.1 Using Command Line

3.1.1 Synopsys

```
convmgr [-h]
(-spreadsheet excel-file | -template | -suggest_pattern_groups STIL REPOSITORY [STIL REPOSITORY ...] )
[-import_pins STIL REPOSITORY [STIL REPOSITORY ...] ]
[-check]
[-keep_stil_files]
[-target TARGET]
[-workdir DIRECTORY]
[-clear]
[-licwait]
```

3.1.2 Command Line Option Description

The following are available command line options of ConvMgr :

-h	Display help message and exit.
-spreadsheet	Specify the path to the spreadsheet in XLS/XLSX format (Microsoft Excel).
-template	Generate empty spreadsheet template.
-target	Specify platform target.
-import_pins	Import pins from given STIL file/dir.
-suggest_pattern_groups	Suggest pattern groups based on specified STIL file/directories.
-check	Check project.
-keep_stil_files	Keep intermediate STIL files.
-clear	Clear output device and reference directories.
-licwait	wait for the license, if not available.

Notes

- When using the **-template** option, the **-target** option must also be used.
- Valid platform target values are 93000DD, 93000SD, 93000SS, 93000SMT8.

3.1.3 An Example of the Command Line

```
>convmgr -spreadsheet input.xls
```

3.1.4 An Example of Template Generation

```
> convmgr -template -target 93000DD
```

Using the template command line argument will generate an empty spreadsheet file and a flow skeleton file for the specified platform.

Chapter 4

Generating Advantest V93000 Output from STIL

4.1 Test Program Description Spreadsheet

The Test Program Description Spreadsheet is a Microsoft Excel spreadsheet constructed from the following sheets:

- **Global sheet**, which provides certain global settings, such as STIL patterns location (Section 4.1.1).
- **TestList sheet**, which provides testsuite description and conversion options (Section 4.1.2).
- **Conversion Setup sheet**, which defines sets of conversion options (Section 4.1.3).
- **TestSuite Flags sheet**, which specifies settings configurations to be used by test suites (Section 4.1.4).
- **TestMethod sheet**, which defines testmethod names and parameters, to be used by the **TestList** sheet (Section 4.1.5).
- **Pins sheet**, which specifies pin names, parameters and channels (Section 4.1.6).
- **Groups sheet**, which defines signal group names and members (Section 4.1.7).
- **PortConfig sheet**, which defines single and multiport configurations and port parameters (Section 4.1.8).
- **Ports sheet**, which defines port names and port members (Section 4.1.7).
- **Bins sheet**, which specifies bin numbers and their parameters (Section 4.1.9).
- **TestFlow Flags sheet**, which specifies the setup for the entire test flow (Section 4.1.10).
- **TestFlow Variables sheet**, which defines Test Program variable declarations and initial values (Section 4.1.11).
- **Tests sheet**, which specifies test limits for tests (Section 4.1.12).
- **Utility Purposes sheet**, which defines settings data for groups of utility lines (Section 4.1.13).

Note that in order to omit a whole row from being read and used during conversion by ConvMgr in each of the sheets, add a # character at the beginning of the leftmost cell of that row.

4.1.1 Global Sheet

The **Global** sheet allows the user to configure global settings related to the entire conversion and assembly process.

Source directory

Path to the STIL source directory.
(Default: ./stil)

Error on Duplicated Source

When this option is set to 0, ConvMgr will allow the use of the same pattern in different tests, and will add a prefix to the pattern name if necessary. Using the same pattern in more than one group with this option set to 1 will result in an error.
(Default: 0)

Device Directory

The *Advantest V93000* output device directory to be generated or updated.
(Default: ./output)

Reference Directory

Path to the output reference directory (see Section 4.3.2).
(Default: "./reference")

Technology File

Path to the SmarTest technology source file ConvMgr will use for *Advantest V93000*. If not specified, ConvMgr will create a technology with default values.

Levels File

Path to the lvi file ConvMgr will use for *Advantest V93000*.

Flow Skeleton File

Path to a TestFlow Skeleton text File (see Section 4.2).

TestTable File

Path to a test table sheet containing test limits. The tests configurations may also be referenced through the Tests sheet (see Section 4.1.12).

Target

Target ATE Platform. Valid values are: 93000SS, 93000DD, 93000SD

Device Name

Project name, serves as a base name for all named test program blocks.
(Default: "project")

SmarTest Release

SmarTest revision number.
(Default: "7.2")

Comment

Write user comment in the log file.

Source Directory	./stil
Error on Duplicated Source	0
Device Directory	./output
Reference Directory	./reference
Technology File	./technology
Levels File	source.lvl
Flow Skeleton File	template_flow.tf
Test Table File	limits.csv
Target	93000DD
Device Name	project
SmarTest Release	7.2
Comment	Comment Example

Figure 4.1: Global Sheet *Advantest V93000*

4.1.2 TestList Sheet

The **TestList** sheet provides a central entry point of information for the STIL patterns conversion and test program assembly. Rows sharing the same testsuite name represent one testsuite to be generated in the final testflow, while the order they are listed in the **TestList** sheet will determine their inner order in the mentioned testsuite.

Source

Name of the input STIL file, expected to be found in the **Input STIL directory** as set in the **Global** sheet.

Conversion Mode

Describes how the pattern should be translated. Possible values are: Force, Skip, Make, Reuse, Ignore, and Placeholder. The conversion mode is interpreted as follows:

- **Force:** Always translate the pattern to *Advantest V93000*.
- **Skip:** Skip pattern translation, but include the testsuite in the final testflow.
- **Make:** Translate the pattern only if the input files or spreadsheet were modified in a way that might affect conversion results or the pattern was never translated.
- **Reuse:** Allow reusing an existing testsuite with the option of changing other tests properties (like timing or levels). To reuse a testsuite, specify its name in the Source column.
- **Ignore:** Completely ignore the pattern and exclude it from the testflow.

- **Placeholder:** Include a dummy test in the final testflow to be manually customized by the user. It is the user's responsibility to verify the test program integrity when using this Mode.

Conversion Setup

Specifies the name of conversion options set from the **Conversion Setup** sheet (see Section 4.1.3).

Pattern Group

The **Pattern Group** name is used as a constraint, in order to make sure all the patterns sharing the same pattern group will use the same values for conversion setup, timing, levels, and port configuration.

TestSuite

Specifies testsuite name. Multiple tests can be integrated into the same testsuite by using the same testsuite name for each of these tests. This is a mandatory field.

Flow Anchor

Specifies which TestFlow skeleton Flow Anchor the testsuite will be assigned to (see Section 4.2).

TestSuite Flags

Specifies setup flags configuration from **TestSuite Flags** sheet to be used by ConvMgr on the testsuite (see Section 4.1.4).

SW Bin

Selects bin from the corresponding **Bins** sheet (see Section 4.1.9). If not specified, `run()` statement will be created for this testsuite in the output flow instead of `run_and_branch()`. If **multi** is specified, `multi_bin` statement will be generated instead of `stop_bin` statement.

Levels

Selects DC levels **EQNSET** index to be used by the testsuite. The specified levels context is the one to be used in the **levels master file**.

Timing

Selects timing **EQNSET** index (for the case of *single port*) or **SPECIFICATION** name (for the case of *multiport*) to be used by the testsuite. The specified timing context is the one to be used in the **timing master file**.

Port Config

Selects port configuration setup from the corresponding **PortConfig** sheet (see Section 4.1.8).

TestMethod

Selects test method definition from the **TestMethod** sheet (see Section 4.1.5).

Burst Name

Specifies the name of the output burst label. if left empty ConvMgr will autogenerate one. When specifying the same burst name for multiple tests, ConvMgr will create a combined burst for all the included test, which must also share the same testsuite.

Source	Conversion Mode	Conversion Setup	Pattern Group	TestSuite	Flow Anchor	TestSuite Flags	SW Bin	Levels	Timing	Port Config	Testmethod	Burst Name
input1.stil	Force	CONV1	G1	TS1	Anchor1	flags1	11	20	SPEC	PC1	FUNC	B1
input2.stil	Make	CONV2	G1	TS2	Anchor1	flags1	22	20	SPEC	PC1	FUNC	B2
input3.stil	Ignore	CONV3	G2	TS3	Anchor2	flags2	33	4	7	PC2	PINCON	B3

Figure 4.2: Testlist Sheet *Advantest V93000*

4.1.3 Conversion Setup sheet

Conversion Setup sheet specifies setup parameters to be used for the translation of STIL patterns. The top row specifies conversion option names, which are linked to **Conversion Setup** column from the **TestList** sheet (see Section 4.1.2). Selecting a specific option name selects the set of STILReader setup parameters listed in the same column. A special flag named `force_z_instead_of_x` may be used. Adding this flag and setting it to 1 will replace all the X states for InOut signals to Z.

The below STILReader setup parameters cannot be used in ConvMgr:

- target
- workdir
- project_name
- stil_file_name_as_pattern_name
- stil_file_name_as_project_name
- exclude_signals
- generate_timing
- generate_patterns
- pin_based_tim
- scan_multiport
- stable_multiport
- use_lvl_groups
- use_eqfs

Notes

- If a parameter is not specified, the STILReader tool default value will be used instead.
- To create a complete list of possible setup parameters, generate a template file for STILReader

Flag Name	CONV1	CONV2
vector_comment_source	C++,C	Annotation
dd_tester_model	PS400	"PS800"
gzip_patterns		0
force_edge_strobe		0

Figure 4.3: Conversion Setup Sheet *Advantest V93000***4.1.4 TestSuite Flags**

TestSuite Flags sheet provides a list of test suite flags that may be assigned for each of the testsuites used by ConvMgr. To assign test suite flags for a testsuite, add a new column to this sheet (or overwrite an existing one).

1. Write a unique name for this setup in the first row.
2. For each row, assign a value for the test suite flag according to the description, or leave the cell empty if the test suite flag is not needed for the current setup configuration.
3. In the TestList sheet (see Section 4.1.2), Choose the name given for this setup configuration in the TestSuite Flags column, to assign the setup values for the testsuite.

Local Flag	flags1
bypass	1
set_pass	0
set_fail	1
hold	
hold_on_fail	0
output_on_pass	1
output_on_fail	0

Figure 4.4: TestSuite Flags Sheet *Advantest V93000*

4.1.5 TestMethod Sheet

TestMethod sheet defines the testmethod **Action Name** and testmethod parameters. The sheet assumes UTM mode is used in the testflow. TestMethod column provides a link between TestList (see Section 4.1.2) and TestMethod sheets.

TestMethod	Action Name	Name1	Value1	Name2	Value2
PINCON	MyTMLib.ClassDcTest.myM	pinlist	@	forceVoltageLow	0.0[V]
FUNC	dc_tml.DcTest.Leakage	pinlist	ALL_DIGITAL	ppmuPrecharg	OFF

Figure 4.5: TestMethod Sheet *Advantest V93000*

4.1.6 Pins sheet

Pins sheet allows specifying channel allocation for the device signals and default states for virtual pins. Each pin has a row with **Pin Name**, **Type**, **Default State**, and **Channels** per site.

Valid pin types are: I,O,IO,DPS.

Default State specifies default fixed state for virtual pins. Valid default state are: 0,1,X. If the pin is defined in the pattern the default states are ignored.

In order to define more sites, new columns must be inserted on the right hand side of the sheet. Note that STIL pins not mentioned in this sheet will be excluded from conversion.

Pin Name	Pin Type	Default State	Site1	Site2
Data1	IO		10101	30101
Data2	DPS		10102	30102
clk1	IO	0	10103	30103
clk2	IO	0	10104	30104
A0	I		10105	30105
A1	I		10106	30106
B0	IO	X	10107	30107
B1	IO	X	10108	30108
Vdd	DPS		11	12

Figure 4.6: Pins Sheet *Advantest V93000*

4.1.7 Ports Sheet and Groups Sheet

Ports sheet and **Groups** sheet contain a list of pin members for each object. Member names are delimited by a comma.

Please note that Groups and Ports cannot share the same name.

The information from these sheets and from the **Pins** sheet presented below, are reflected directly in the pin configuration file of the device.

Port Name	Port Pins
pData	Data1,Data2
pClock	clk1,clk2
pA	A0,A1
pB	B0,B1

(a) Ports Sheet

Group Name	Group Pins
g1	Data1,clk1,A1,B1
g2	Data2,clk2,A0,B0
gA	A0,A1
gB	B0,B1

(b) Groups Sheet

Figure 4.7: Ports Sheet and Groups Sheet *Advantest V93000*

4.1.8 PortConfig Sheet

PortConfig sheet is used to configure multiport and single port bursts. For every port setup, each port requires a separate row with port name, xmode factor, wavetable name and other port parameters. All rows having the same **PortConfig** define together the complete configuration.

Single port configurations are defined by using @ as the **Port Name**. The **PortConfig** column provides a link between to the associated testsuites in the **TestList** sheet (see Section 4.1.2).

PortConfig	PortName	X Mode	Wavetable Name	Additional Parameters
ACSCAN	pData	2	wtb1	card=PS800
ACSCAN	pClock	4	wtb1	card=PS800, master_clock=mclk1
MBIST	pA	2	wtb3	
MBIST	pB	1		
Single Port X1	@		wtbSP1	
Single Port X2	@	2	wtbSP2	
ACSCAN2	pData	2	wtb2	card=PS800, master_clock=mclk1, sync_group=syn1
ACSCAN2	pClock	4	wtb2	card=PS800, master_clock=mclk2

Figure 4.8: PortConfig Sheet *Advantest V93000*

4.1.9 Bins sheet

Bins sheet specifies *stop bins* for testsuites. The **Soft BIN** column provides the link between the **TestList** sheet (see Section 4.1.2) and the **Bins** sheet. Each row defines the **stop bin** declaration and contains **Soft BIN** and **Hard BIN** indexes and matching **Labels**.

Please note that **Soft BIN** index 1 is reserved for the good bin and cannot be used.

Soft BIN	Soft Label	Hard BIN	Hard Label
10	soft1	10	hard1
11	soft2	10	hard1
12	soft3	10	hard1
13	soft4	10	hard1
14	soft5	10	hard1
15	soft6	20	hard2

Figure 4.9: Bins Sheet *Advantest V93000*

4.1.10 TestFlow Flags

TestFlow Flags sheet specifies the *Advantest V93000* global settings flags for the entire test flow.

Flag	Value
report_to_file	0
report_to_printer	0
datalog_to_file	1
datalog_to_printer	0
datalog_to_report_win	1
datalog_formatter	0
datalog_sample_size	1
graphic_result_display	1

Figure 4.10: TestFlow Flags Sheet *Advantest V93000*

4.1.11 TestFlow Variables

TestFlow Variables sheet specifies the variables names, types and initial values used in the *Advantest V93000* test program.

Name	Type	Initial Value
label1	STRING	test_start
varX	DOUBLE	5.0
edgeA	DOUBLE	12.34
Init	USERFLAG	1

Figure 4.11: TestFlow Variables Sheet *Advantest V93000*

4.1.12 Tests

Tests sheet specifies the lower and upper limits. Tests configuration can also be imported from a file using the Test Table file parameter in the **Global** sheet (see Section 4.1.1). When importing a tests sheet file, ConvMgr will combine the data from both the local tests sheet, and the imported tests sheet.

Testsuite	Test Name	Test Number	Lower Limit	Lower Symbol	Upper Symbol	Upper Limit	Unit
TS1	PE1	1	200	<	<=	1000	mA
TS2	PE2	2	0.5	<	<=	3	V
TS3	PE3	3	1.2	<	<=	2.8	V

Figure 4.12: Tests Sheet *Advantest V93000*

4.1.13 Utility Purposes

Utility Purposes sheet specifies setting data for utility line groups. The first column describes the group name, the second column determines the channel for the group, and the next columns will set the data for each site (i.e 0, 1 or X).

Name	Channel	Site1	Site2
relay0	101	1	0
relay1	102	1	1
relay2	103	0	1
trigger	105	X	X

Figure 4.13: Utility Purposes Sheet *Advantest V93000*

4.2 TestFlow Skeleton

This file will include the TestFlow structure instructions for ConvMgr. For each section in the structure, the user may leave the default angled brackets expressions (e.g. <convmgr_information>, <convmgr_testmethods> etc.), thus ConvMgr will replace the expression according to the input spreadsheet. The user may also replace or add information as needed.

A template skeleton file may be created using the command line template generation (see Section 3.1).

```
----- TestFlow Skeleton input Example -----
1  information
2    <convmgr_information>
3  end
4  -----
5  declarations
6    myVar = "String";
7  end
8  -----
9  flags
10   <convmgr_flags>
11   report_to_printer = 0;
12  -----
----- OutPut testflow Advantest V93000 Example -----
1  }
2  information
3    device_revision = "1";
4  end
5  -----
6  declarations
7    myVar = "String";
8  end
9  -----
10 flags
11   report_to_file = 1;
12   report_to_printer = 0;
13  -----
```

In the above example, the `<convmgr_flags>` special string was replaced with all the specified TestFlow Flags and their values. In that case, assume that the TestFlow Flags sheet includes a single flag, 'report_to_file', with the value 1.

The test_flow section requires special handling. This section may include special commands such as If/Then, Repeat, While and For Loops. The user will assign *Anchor*s for Test Suites in the TestList sheet (see Section 4.1.2). The same *Anchor* may be assigned to more than one Test Suite. ConvMgr will read the test_flow and create the Advantest V93000 TestFlow according to the given structure, while replacing each *Anchor* with the Test Suites assigned to it in a *run_and_branch* block. If the *Anchor* is used as a parameter for a function, it will be replaced with its assigned Test Suite without creating a new block. Please note that only one Test Suite can be assigned to a *Anchor* if used as a parameter for a function. The order of the *Anchor*s will determine the order of the Test Suites in the TestFlow. If multiple Test Suites are assigned to a single *Anchor*, the order of the Test Suites assigned to this *Anchor* in the TestList sheet will determine the inner sorting.

Special *Anchor*s may also be used:

- **<ALL>** : Execute all the Test Suites.
- **<ALL UNUSED>** : Execute all the Test Suites that are not associated with any *Anchor*.

For the following test_flow example, assume TestSuite1 and TestSuite2 are assigned to Anchor1, and TestSuite3 is assigned to Anchor2.

```
_____ TestFlow Skeleton test_flow Section Example _____  
1  <Anchor1>  
2  run_and_branch(<Anchor2>)  
3  then  
4  {  
5  }  
6  else  
7  {  
8      stop_bin "22","simple2 failed",,bad,noreprobe,red,10,over_on;  
9  }  
10 <Anchor1>  
_____
```

_____ Output Advantest V93000 test_flow Example _____

```
1  run_and_branch(TestSuite1)
2  then
3  {
4  }
5  else
6  {
7      stop_bin "11", "soft1", ,bad,noreprobe,red,10,over_on;
8  }
9  run(TestSuite2);
10 run_and_branch(TestSuite3)
11 then
12 {
13 }
14 else
15 {
16     stop_bin "22", "simple2 failed", ,bad,noreprobe,red,10,over_on;
17 }
18 run_and_branch(TestSuite1)
19 then
20 {
21 }
22 else
23 {
24     stop_bin "11", "soft1", ,bad,noreprobe,red,10,over_on;
25 }
26 run(TestSuite2);
```

4.3 Output Files

ConvMgr produces the *Advantest V93000* test program directory as well as reference and logs directories.

4.3.1 Test Program Directory

ConvMgr result is a single *Advantest V93000* test program directory.

Testflow

ConvMgr generates a single flat testflow file. The file includes the execution structure of the test. The `test_flow` section holds the testsuites, which appear in the order matching original rows order in the **TestList** sheet. Special instructions for creation of the testflow file may be given using an external skeleton file (see Section [4.2](#)).

Timing and Patterns

ConvMgr uses master files to assemble timing and patterns. There is a single generated master file for each of the **timing** and **patterns** configuration files. Alongside the master files ConvMgr generates subdirectories which contain pattern group related components.

The **timing master file** components are differentiated by EQNSET index or SPECIFICATION name of the **Timing** column in the **TestList** sheet (see Section 4.1.2). The timing components are the result of incremental generation based on the source STIL patterns for each pattern group. The **pattern master file** contains the names and locations of the binary pattern files.

Pin Configurations

ConvMgr generates a commands file (with a cha suffix), that defines the pin configurations for the test. The resulting configuration instructions are based on multiple sheets, including the **Pins** sheet for pin name, type and site (Section 4.1.6), **Ports** and **Groups** sheets for pin assignment for ports and groups (Section 4.1.7), and also **Utility Purposes** for utility lines channels (Section 4.1.13).

4.3.2 Reference

In addition to the test program directory, ConvMgr produces reference directories used for Make mode and debugging, **Timing** and **Tests**. These directories hold intermediate results used by ConvMgr to create the final test program. When using Make mode conversion option, the reference directories are examined for modifications. Therefore changing/deleting files from the reference directory may force pattern conversion for sources that were already processed, instead of using up-to-date conversion results.

The **Timing** directory includes the *Advantest V93000* timing specification file and a timing combination file, which holds incremental timing information of all the testsuites. The **Tests** directory organizes the testsuites into subdirectories, with a subdirectory for each testsuite according to the **TestList** sheet (similar to the testsuite directory in the output of ConvMgr). Each subdirectory includes setup files, output directory and conversion log.

Chapter 5

Generating Advantest V93000SMT8 Output from STIL

5.1 Test Program Description Spreadsheet

The Test Program Description Spreadsheet is a Microsoft Excel spreadsheet constructed from the following sheets:

- **Global sheet**, which provides certain global settings, such as STIL patterns location (Section [5.1.1](#)).
- **TestList sheet**, which provides testsuite description and conversion options (Section [5.1.2](#)).
- **Conversion Setup sheet**, which defines sets of conversion options (Section [5.1.3](#)).
- **TestMethod sheet**, which defines testmethod names and parameters, to be used by the **TestList** sheet (Section [5.1.4](#)).
- **Pins sheet**, which specifies pin names, parameters and channels (Section [5.1.5](#)).
- **Groups sheet**, which defines signal group names and members (Section [5.1.6](#)).
- **PortConfig sheet**, which defines single and multiport configurations and port parameters (Section [5.1.7](#)).
- **Ports sheet**, which defines port names and port members (Section [5.1.6](#)).
- **Bins sheet**, which specifies bin numbers and their parameters (Section [5.1.8](#)).

Note that in order to omit a whole row from being read and used during conversion by ConvMgr in each of the sheets, add a # character at the beginning of the leftmost cell of that row.

5.1.1 Global Sheet

The **Global** sheet allows the user to configure global settings related to the entire conversion and assembly process.

Source directory

Path to the STIL source directory.
(Default: ./stil)

Error on Duplicated Source

When this option is set to 0, ConvMgr will allow the use of the same pattern in different tests, and will add a prefix to the pattern name if necessary. Using the same pattern in more than one group with this option set to 1 will result in an error.
(Default: 0)

Device Directory

The *Advantest V93000SMT8* output device directory to be generated or updated.
(Default: ./output)

Reference Directory

Path to the output reference directory (see Section 5.3.2).
(Default: "./reference")

Levels Directory

Path to the directory containing lvl files ConvMgr will use for *Advantest V93000SMT8*.

Flow Skeleton File

Path to a TestFlow Skeleton text File (see Section 5.2).

Target

Target ATE Platform. Value should be 93000SMT8.

Device Name

Project name, serves as a base name for all named test program blocks.
(Default: "project")

SmarTest Release

SmarTest revision number.
(Default: "8.0.0")

Comment

Write user comments in the log file.

Source Directory	./stil
Error on Duplicated Source	0
Device Directory	./output
Reference Directory	./reference
Levels Directory	./Levels
Flow Skeleton File	template_flow.flow
Target	93000SMT8
Device Name	project
SmarTest Release	8.0.0
Comment	Test1

Figure 5.1: Global Sheet *Advantest V93000SMT8*

5.1.2 TestList Sheet

The **TestList** sheet provides a central entry point of information for the STIL patterns conversion and test program assembly. Rows sharing the same testsuite name represent one testsuite to be generated in the final testflow, while the order they are listed in the **TestList** sheet will determine their inner order in the mentioned testsuite.

Source

Name of the input STIL file, expected to be found in the **Input STIL directory** as set in the **Global** sheet.

Conversion Mode

Describes how the pattern should be translated. Possible values are: Force, Skip, Make, Reuse, Ignore, and Placeholder. The conversion mode is interpreted as follows:

- **Force:** Always translate the pattern to *Advantest V93000SMT8*.
- **Skip:** Skip pattern translation, but include the testsuite in the final testflow.
- **Make:** Translate the pattern only if the input files or spreadsheet were modified in a way that might affect conversion results or the pattern was never translated.
- **Reuse:** Allow reusing an existing testsuite with the option of changing other tests properties (like timing or levels). To reuse a testsuite, specify its name in the Source column.
- **Ignore:** Completely ignore the pattern and exclude it from the testflow.
- **Placeholder:** Include a dummy test in the final testflow to be manually customized by the user. It is the user's responsibility to verify the test program integrity when using this Mode.

Conversion Setup

Specifies the name of conversion options set from the **Conversion Setup** sheet (see Section [5.1.3](#)).

Pattern Group

The **Pattern Group** name is used as a constraint, in order to make sure all the patterns sharing the same pattern group will use the same values for conversion setup, timing, levels, and port configuration.

Suite

Specifies testsuite name. Multiple tests can be integrated into the same testsuite by using the same testsuite name for each of these tests. This is a mandatory field.

Flow Anchor

Specifies which TestFlow skeleton Flow Anchor the testsuite will be assigned to (see Section 5.2).

SW Bin

Selects a bin from the corresponding **Bins** sheet (see Section 5.1.8).

Levels

Selects DC levels file to be used by the testsuite. Tests from the same **pattern group** cannot have different values in the **levels** column. When not specified, ConvMgr will automatically fill the value according to the test's **pattern group**.

Timing

Tests sharing the same value in this column will use a common incremental timing file for conversion. Tests from the same **pattern group** cannot have different values in the **timing** column. When not specified, ConvMgr will automatically fill the value according to the test's **pattern group**.

Port Config

Selects port configuration setup from the corresponding **PortConfig** sheet (see Section 5.1.7).

TestMethod

Selects the test method definition from the **TestMethod** sheet (see Section 5.1.4).

Burst Name

Specifies the name of the output burst label. **Burst Name** should be unique. if left empty ConvMgr will autogenerate one.

Source	Conversion Mode	Conversion Setup	Pattern Group	Suite	Flow Anchor	Sw Bin	Levels	Timing	Port Config	Testmethod	Burst Name
input1.stil	Force	CONV1	PatGroup1	Suite1	Anchor1	10	levels1.spec	1	config1	TM1	B1
input2.stil	Make	CONV2	PatGroup2	Suite2	Anchor2	20	levels2.spec	2	config2	TM1	B2

Figure 5.2: Testlist Sheet *Advantest V93000SMT8*

5.1.3 Conversion Setup sheet

Conversion Setup sheet specifies setup parameters to be used for the translation of STIL patterns. The top row specifies conversion option names, which are linked to the **Conversion Setup** column of the **TestList** sheet (see Section 5.1.2).

Selecting a specific option name selects the set of STILReader setup parameters listed in the same column.

The below STILReader setup parameters cannot be used in ConvMgr:

- target
- workdir
- project_name
- stil_file_name_as_pattern_name
- stil_file_name_as_project_name
- exclude_signals
- virtual_signals
- generate_timing
- generate_patterns
- pin_based_tim
- spec_layout

Notes

- If a parameter is not specified, the STILReader tool default value will be used instead.
- To create a complete list of possible setup parameters, generate a template file for STILReader

flag name	CONV1	CONV2
vector_comment_source	C++,C	C++,C
workdir	'output1'	'output2'
pattern_comment	'Using conversion manager'	
rename_signals		{ 'data': 'data_new' }

Figure 5.3: Conversion Setup Sheet *Advantest V93000SMT8*

5.1.4 TestMethod Sheet

TestMethod sheet defines the testmethod **Action Name** and testmethod parameters. The TestMethod column in the TestList sheet (see Section 5.1.2) provides a link between the TestList sheet and the TestMethod sheet.

TestMethod	Action Name	Name1	Value1	Name2	Value2
TM1	ac_tml.AcTest.FunctionalTest	param1	0	param2	"string"
TM2	ac_tml.AcTest.action	X	5	Y	"string2"

Figure 5.4: TestMethod Sheet *Advantest V93000SMT8*

5.1.5 Pins sheet

Pins sheet allows specifying channel allocation for the device signals and default states for virtual pins. Each pin has a row with **Pin Name**, **Type**, **Default State**, and **Channels** per site.

Valid pin types are: I,O,IO,DPS. Currently Analog and RF types are not supported.

Default State specifies default fixed state for virtual pins. Valid default state are: 0,1,X. If the pin is defined in the pattern the default states are ignored. You may also gang or stack channels in the **site** column using the operator + (stack) or operator | (gang) (see example in Figure 5.5).

ConvMgr will automatically generate default timing values for pins which are not defined in the source files but are used in the **Pins** sheet (such as virtual pins).

In order to define more sites, new columns must be inserted on the right hand side of the sheet. Note that STIL pins not mentioned in this sheet will be excluded from conversion.

Pin Name	Pin Type	Default State	Site1	Site2
data_1	InOut		10103	10203 10207
data_2	InOut		10105	10205 + 10209
virtual1	InOut	0	10109	10409 + 10509 + 10609
virtual2	In	1	10301	10401 10501 10601
virtual3	Out	X	10303	10403

Figure 5.5: Pins Sheet *Advantest V93000SMT8*

5.1.6 Ports Sheet and Groups Sheet

). **Ports** sheet and **Groups** sheet contain a list of pin members for each object. Member names are delimited by a comma.

Please note that Groups and Ports cannot share the same name.

The information from these sheets and from **Pins** sheet presented below, are reflected directly in the pin configuration file of the device.

Port Name	Port Pins
P1	data0,data1,data2,data3
P2	data4,data5,data6,data7

(a) Ports Sheet

Group Name	Group Pins
group1	data0,data1
group2	data2,data3
group3	data5,data6,data7

(b) Groups Sheet

Figure 5.6: Ports Sheet and Groups Sheet *Advantest V93000SMT8*

5.1.7 PortConfig Sheet

PortConfig sheet is used to configure multiport and single port bursts. For every port setup, each port requires a separate row with port name, xmode factor, wavetable name and other port parameters. All rows having the same **PortConfig** define together the complete configuration.

Single port configuration are defined by using @ as **Port Name**. **PortConfig** is selected in the Port Config column in the **TestList** sheet (see Section 5.1.2).

PortConfig	PortName	X Mode	Wavetable Name	Additional Parameters
config1	P1	1	t1	card=PS8000
config1	P2	2	t2	sync_ctim = 0
single_port1	@	1	t3	

Figure 5.7: PortConfig Sheet *Advantest V93000SMT8*

5.1.8 Bins sheet

Bins sheet specifies *stop bins* for testsuites. The **Soft BIN** column provides the link between the **TestList** sheet (see Section 5.1.2) and the **Bins** sheet. Each row defines **stop bin** declaration and contains **Soft BIN** and **Hard BIN** indexes and matching **Labels**.

Please note that **Soft BIN** index 1 is reserved for the good bin and cannot be used.

Software Bin	Software Bin Name	Hardware Bin	Hardware Bin Name
10	soft1	10	hard1
20	soft2	10	hard1
30	soft3	20	hard2

Figure 5.8: Bins Sheet *Advantest V93000SMT8*

5.2 Flow Skeleton

This file will include the flow structure instructions for ConvMgr. The flow block may include special commands such as if, While, and for. The user will assign *Flow Anchors* for Test Suites in the TestList sheet (see Section 5.1.2). A *Flow Anchor* may be assigned to more than one Test Suite. ConvMgr will read the test_flow and create the Advantest V93000SMT8 TestFlow according to the given structure, while replacing each *Flow Anchor* with the Test Suites assigned to it with a *execute()* command. The order of the *Flow Anchor* will determine the order of Test Suites in the flow. If multiple Test Suites are assigned to a single *Flow Anchor*, the order of the Test Suites assigned to this *Flow Anchor* in the TestList sheet will determine the inner sorting. Special *Flow Anchors* may also be used:

- **<convmgr_execute>** : Default Flow execution.
- **<Flow Anchor>.member** : Use member instead of execute() function for the assigned Test Suite. if more than one Test Suite is assigned to this *Flow Anchor*, only the first one will be used.
- **<ALL>** : Execute all the Test Suites.
- **<ALL UNUSED>** : Execute all the Test Suites that are not associated with any *Flow Anchor*.

A template skeleton file may be created using the command line template generation (see Section 3.1).

For the following test_flow example, assume Suite1 and Suite3 are assigned to Anchor1 Suite2 is assigned to Anchor2, and Suite4 is not assigned to any anchor. The suites defined in the spreadsheet will replace the **<convmgr_suites>** special string, while using the relevant information from their corresponding Testmethod in the TestMethod sheet (see Section 5.1.4).

```

_____ Flow Skeleton Input File Example _____
1 flow <device_name> {
2   setup {
3     <convmgr_suites>
4   }
5   execute {
6     if (<Anchor2>.foo()) {
7       <ALL>
8     }
9     else {
10      <ALL UNUSED>
11    }
12    <Anchor1>
13  }
14 }
15
_____

```

_____ Output Advantest V93000SMT8 Flow Example _____

```
1 flow project {
2   setup {
3     suite Suite1 calls ac_tml.AcTest.FunctionalTest {
4       measurement.specificationName = setupRef(project.suites.Suite1.Suite1);
5       measurement.operatingSequenceName = setupRef(project.suites.Suite1.B1);
6     }
7     suite Suite2 calls ac_tml.AcTest.FunctionalTest {
8       measurement.specificationName = setupRef(project.suites.Suite2.Suite2);
9       measurement.operatingSequenceName = setupRef(project.suites.Suite2.B2);
10    }
11    suite Suite3 calls ac_tml.AcTest.FunctionalTest {
12      measurement.specificationName = setupRef(project.suites.Suite3.Suite3);
13      measurement.operatingSequenceName = setupRef(project.suites.Suite3.B3);
14    }
15    suite Suite4 calls ac_tml.AcTest.FunctionalTest {
16      measurement.specificationName = setupRef(project.suites.Suite4.Suite4);
17      measurement.operatingSequenceName = setupRef(project.suites.Suite4.B4);
18    }
19  }
20  execute {
21    if (Suite2.foo()) {
22      Suite1.execute();
23      Suite2.execute();
24      Suite3.execute();
25      Suite4.execute();
26    }
27    else {
28      Suite4.execute();
29    }
30    Suite1.execute();
31    Suite3.execute();
32  }
33 }
```

5.3 Output Files

ConvMgr produces the *Advantest V93000SMT8* test program directory as well as reference and logs directories.

5.3.1 Test Program Directory

ConvMgr output result is a single *Advantest V93000SMT8* test program directory. The flow of the test program starts with a prebinding call which creates the bins table, followed by the execution of the main testflow.

PreBind Flow

The first phase of the test program flow is the execution of an external file located in the **testmethod** directory, inside the project source directory. This process creates a prebinding of the software and hardware bins table, which will be used by the test program. The definitions of the bins is done in the **Bins** sheet (see Section 5.1.8).

Main Testflow

By default, the test program main flow includes the execution of all the suites defined in the **Suite** column in the **TestList** sheet. All the suites are created in the **suites** directory, located inside the project directory, and divided into separate subdirectories for each test suite. A user customized flow may be defined using a predefined skeleton flow file (see Section 5.2).

5.3.2 Reference

In addition to the test program directory, ConvMgr produces reference directories used for Make mode and debugging, **Timing** and **Tests**. These directories hold intermediate results used by ConvMgr to create the final test program. When using Make mode conversion option, the reference directories are examined for modifications. Therefore changing/deleting files from the reference directory may force pattern conversion for sources that were already processed, instead of using up-to-date conversion results.

The **Timing** directory includes the *Advantest V93000SMT8* timing specification file and a timing combination file, which holds incremental timing information of all the testsuites. The **Tests** directory organizes the testsuites into subdirectories, with a subdirectory for each testsuite according to the **TestList** sheet (similar to the testsuite directory in the output of ConvMgr). Each subdirectory includes setup files, output directory and conversion log.

Chapter 6

Additional Capabilities

ConvMgr features additional automated capabilities for facilitating the process of building and updating the test program description spreadsheet.

6.1 Import Pins

In this work mode ConvMgr imports pins, groups and suggests ports from the patterns found in the provided directory into the test program description spreadsheet.

ConvMgr support STIL,WGL,SVF and TDL patterns as input for this mode.

The following command line arguments should be provided:

1. Path to the test program description spreadsheet using `-spreadsheet` parameter. If the spreadsheet does not exist, it will be created.
2. Path to a Test file or a directory containing Test patterns using `-import_pins` parameter.

6.1.1 Example of Import Pin Usage

```
>convmgr -spreadsheet du01_a01.xls -import_pins /home/project/du01/a01/jtag_patterns
```

6.2 Suggest Pattern Groups

In this work mode ConvMgr suggests pattern groups and ports based on the timing from the patterns found in the provided directory into a csv file.

ConvMgr support STIL,WGL,SVF and TDL patterns as input for this mode.

The following command line argument should be provided:

1. Path to a Test file or a directory containing Test patterns using `-suggest_pattern_groups` parameter.

6.2.1 Example of Suggest Pattern Groups Usage

```
> convmgr -suggest_pattern_groups /home/project/du01/a01/jtag_patterns
```

6.3 Check Project

In this work mode ConvMgr checks the project's spreadsheet and sources.

The purpose of this mod is to alert of problems in advance, before performing a long conversion action.

The following command line argument should be provided:

1. Path to the test program description spreadsheet using `-spreadsheet` parameter.

6.3.1 Example of Check Project Usage

```
> convmgr -check -spreadsheet du01_a01.xls
```
